

TCP Client Experiment

TCP Client Experiment

- 1. TCP Overview
- 2. Core Concepts
 - 2.1 TCP Three-Way Handshake
 - 2.2 TCP vs UDP API Comparison
 - 2.3 Three Return Values of `recv()`
- 3. Project Architecture and Flow
 - 3.1 TCP Client
- 4. Hardware Dependencies
- 5. Source Code Analysis — TCP Client
 - 5.1 Macro Configuration
 - 5.2 WiFi Initialization (Review)
 - 5.3 `vTcpClientTask()` — TCP Client Core
- 6. Operation Flow
 - 6.1 Build and Flash Flow
 - 6.2 Testing Method
 - 6.3 Normal Log — TCP Client
- 7. Common Problems

1. TCP Overview

TCP (Transmission Control Protocol) is a **connection-oriented, reliable, byte-stream** transport layer protocol. It establishes connections via three-way handshake, ensuring data arrives in order and intact.

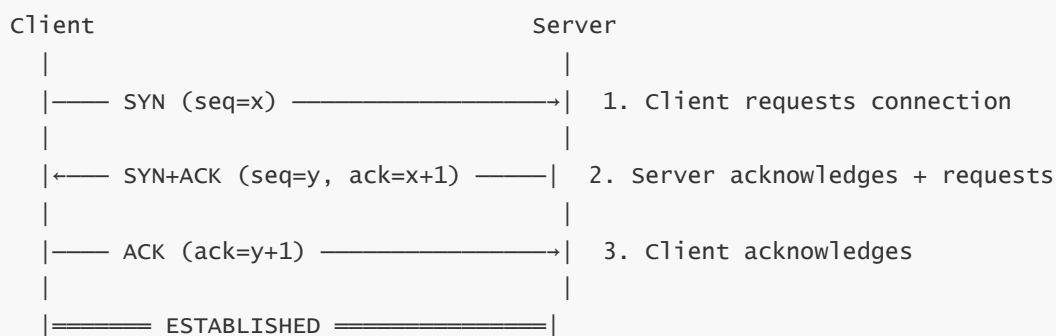
Feature	TCP	UDP
Connection	Three-way handshake	Connectionless
Reliability	Guaranteed delivery, guaranteed order	Not guaranteed
Flow/ Congestion Control	Built-in sliding window, slow start	None
Transmission Mode	Byte stream (no message boundaries)	Datagram (has message boundaries)
Server API	<code>bind</code> → <code>listen</code> → <code>accept</code> → <code>recv/send</code>	<code>bind</code> → <code>recvfrom/sendto</code>
Client API	<code>socket</code> → <code>connect</code> → <code>recv/send</code>	<code>socket</code> → <code>sendto/recvfrom</code>

TCP is the foundation of IoT communications. Whether HTTP web requests, MQTT message push, OTA firmware upgrades, or database remote connections, all rely on TCP's reliable transmission at the bottom. In industrial automation, smart home, and remote monitoring scenarios, devices report sensor data and receive control instructions—all requiring "no data loss, no order disruption"—which is TCP's core value. UDP is fast but unreliable; TCP has the overhead of three-way handshake but gains an **end-to-end acknowledgment and retransmission mechanism**, irreplaceable in most IoT scenarios.

This project contains both TCP client and server routines: the client connects to the server and sends data periodically; the server listens on a port, accepts client connections, sends/receives data, and supports reconnecting after client disconnection.

2. Core Concepts

2.1 TCP Three-Way Handshake



In code, the client calls `connect()` to trigger the handshake, and the server calls `accept()` to wait for handshake completion.

2.2 TCP vs UDP API Comparison

```
// === TCP Server ===
socket(AF_INET, SOCK_STREAM, 0)
bind(fd, addr, len)
listen(fd, 5)
client_fd = accept(fd, ...)
recv(client_fd, buf, len, 0)
send(client_fd, buf, len, 0)

// === TCP Client ===
socket(AF_INET, SOCK_STREAM, 0)
connect(fd, &addr, len)
send(fd, buf, len, 0)
recv(fd, buf, len, 0)

// === UDP Server ===
socket(AF_INET, SOCK_DGRAM, 0)
bind(fd, addr, len)
// No listen
// No accept, use recvfrom directly
recvfrom(fd, buf, len, 0, &addr, &len)
sendto(fd, buf, len, 0, &addr, len)

// === UDP Client ===
socket(AF_INET, SOCK_DGRAM, 0)
// No connect
sendto(fd, buf, len, 0, &addr, len)
recvfrom(fd, buf, len, 0, &addr, &len)
```

2.3 Three Return Values of `recv()`

Return Value	Meaning	Handling
<code>> 0</code>	Received n bytes of data	Process normally
<code>= 0</code>	Peer closed connection normally (sent FIN)	<code>close()</code> and <code>break</code>
<code>< 0</code>	No data available (non-blocking) or error	Ignore with <code>MSG_DONTWAIT</code> ; check <code>errno</code> for others

`recv() == 0` is a TCP-specific "peer closed" signal; UDP has no such concept.

3. Project Architecture and Flow

3.1 TCP Client

```
app_main()
├─ wifi_init()           ← WiFi STA init (wait for IP)
├─
├─ xTaskCreate(vTcpClientTask, ...) ← Create TCP client task
├─ vTcpClientTask():
│   ├─ socket(AF_INET, SOCK_STREAM, 0) → Create TCP socket
│   ├─ connect(192.168.11.183:8080)    → Three-way handshake
│   └─ while(1):
│       ├─ send("ESP32 TCP Client Data")
│       ├─ recv(MSG_DONTWAIT) ← Non-blocking receive
│       ├─ recv==0 → server disconnected, break
│       └─ vTaskDelay(200ms)
```

4. Hardware Dependencies

- **Hardware Connection:** No external wiring, WiFi wireless communication
- **Required Components:** `lwip/sockets`, `esp_wifi`, `esp_netif`, `esp_event`, `nvs_flash`, `FreeRTOS`

5. Source Code Analysis — TCP Client

Full source: `Source Code` → `ESP-IDF` → `5.Network_Protocol` → `4.TCP` → `TCP_Client`

5.1 Macro Configuration

These macros define WiFi connection info and TCP server address. Modify SSID, password, and target IP according to your network environment.

```
#define WIFI_SSID      "Your_WiFi_SSID"      // WiFi SSID
#define WIFI_PASS      "Your_WiFi_Password"  // WiFi password
#define TCP_SERVER_IP  "192.168.11.183"      // Server IP
#define TCP_SERVER_PORT 8080                 // Server port
#define WIFI_TIMEOUT_MS 15000
```

5.2 WiFi Initialization (Review)

The WiFi STA connection code in this experiment (wifi_init_sta(), event callback, NVS init, etc.) is identical to **5.2 WIFI-STA Mode**. `app_main()` does NVS init → WiFi connection to get IP → `xTaskCreate()` for TCP task.

For details, review: 5. Network Protocols and Wireless Communication -> 2. WIFI-STA Mode -> WIFI-STA Mode

5.3 vTcpClientTask() — TCP Client Core

The biggest difference from UDP client: TCP is "make a call before speaking"—`send()` / `recv()` only after `connect()` three-way handshake succeeds. UDP is "send a letter directly"—`sendto()` doesn't need a connection first. Also TCP uses `SOCK_STREAM` (streaming), UDP uses `SOCK_DGRAM` (datagram).

In the send-receive loop, a key judgment—`recv()` returning 0 means the server normally closed the connection (sent FIN package), then `break` out to close socket and delete task. UDP has no this problem because UDP is connectionless, `recvfrom()` never returns 0.

```
void vTcpClientTask(void *pv)
{
    int sock_fd;
    struct sockaddr_in server_addr = {0};
    char send_buf[64] = {0};
    char recv_buf[64] = {0};

    // Create TCP socket
    sock_fd = socket(AF_INET, SOCK_STREAM, 0);
    if (sock_fd < 0) {
        ESP_LOGE(TAG, "Create TCP socket failed");
        vTaskDelete(NULL);
        return;
    }

    // Configure server address and connect
    server_addr.sin_family = AF_INET;
    // Port to network byte order
    server_addr.sin_port = htons(TCP_SERVER_PORT);
    inet_pton(AF_INET, TCP_SERVER_IP, &server_addr.sin_addr.s_addr);

    if (connect(sock_fd, (struct sockaddr*)&server_addr,
```

```

        sizeof(server_addr)) < 0) {
    // Connect failed
    ESP_LOGE(TAG, "Connect TCP server failed");
    close(sock_fd);
    vTaskDelete(NULL);
    return;
}
// TCP handshake complete
ESP_LOGI(TAG, "TCP server connected");

// Send-receive loop
while (1) {
    // Build send data
    sprintf(send_buf, "ESP32 TCP Client Data");
    // Send data
    int ret = send(sock_fd, send_buf, strlen(send_buf), 0);
    if (ret <= 0) {
        // Send failed
        ESP_LOGE(TAG, "Send failed");
        break;
    }
    ESP_LOGI(TAG, "Send: %s", send_buf);
    // Non-blocking receive
    int recv_len = recv(sock_fd, recv_buf, sizeof(recv_buf) - 1,
                        MSG_DONTWAIT);
    if (recv_len > 0) {
        recv_buf[recv_len] = '\0';
        ESP_LOGI(TAG, "Recv: %s", recv_buf);
    }
    else if (recv_len == 0) {
        // Peer closed connection
        ESP_LOGE(TAG, "TCP server disconnected");
        break;
    }

    vTaskDelay(pdMS_TO_TICKS(200));
}

close(sock_fd);
vTaskDelete(NULL);
}

```

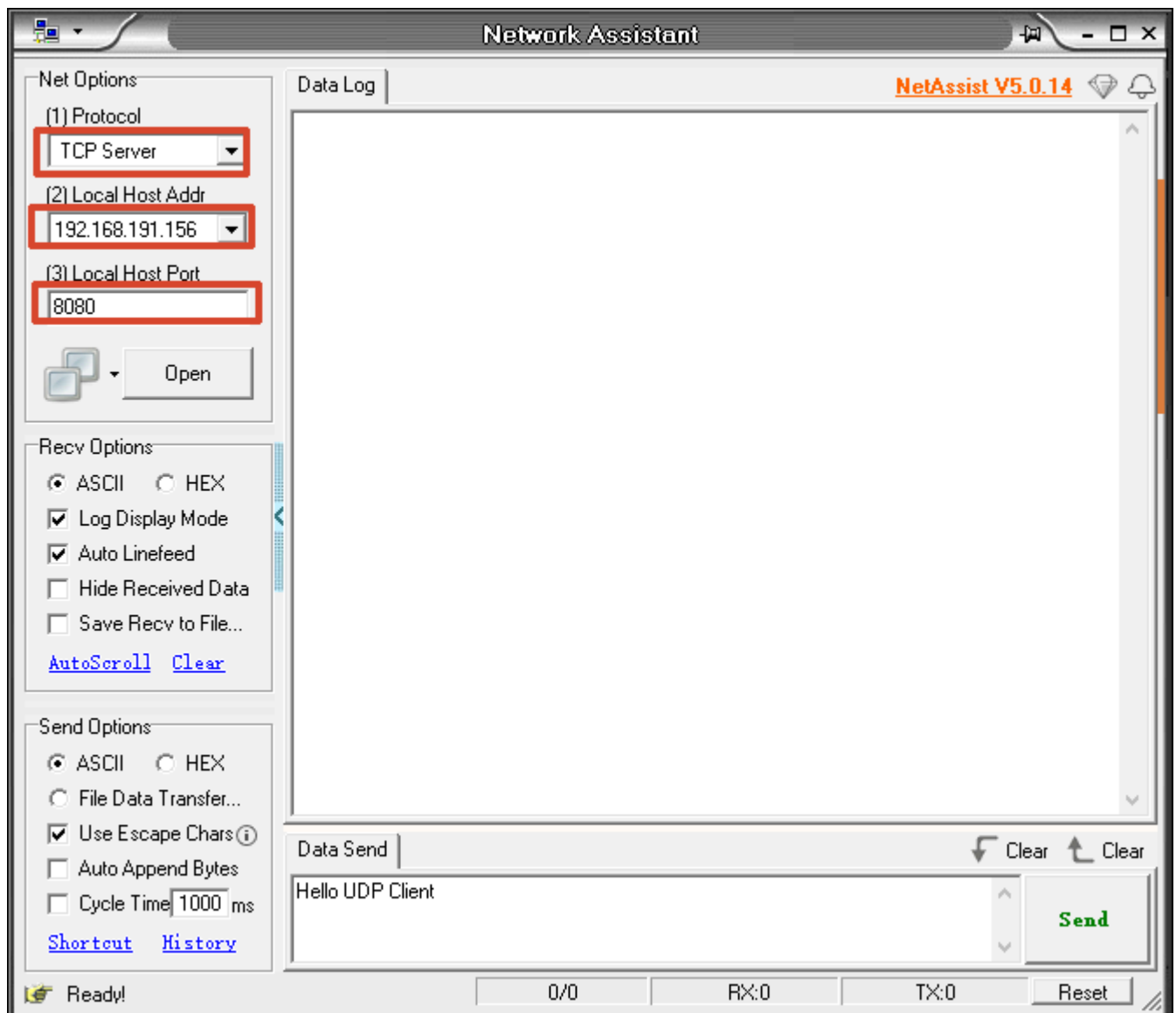
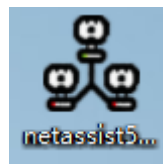
6. Operation Flow

6.1 Build and Flash Flow

ESP-IDF Flash Flow: See [ESP-IDF → 1. Before Development → 3. Build and Flash](#).

6.2 Testing Method

1. Run a network debugging assistant on PC, select TCP Server mode, listen on port 8080.



2. Flash TCP_Client to ESP32. Serial shows connection success → send data → receive reply.

```

I (1704) TCP-CLIENT: Got IP: 192.168.11.181
I (1704) TCP-CLIENT: WiFi connected
I (1704) TCP-CLIENT: TCP socket created
I (1714) main_task: Returned from app_main()
I (1724) wifi:<ba-add>idx:1 (ifx:0, 80:8f:1d:20:9d:f7), tid:0, ssn:
0, winSize:64
I (1724) TCP-CLIENT: TCP server connected
I (1724) TCP-CLIENT: Send: ESP32 TCP Client Data
I (1924) TCP-CLIENT: Send: ESP32 TCP Client Data
I (2124) TCP-CLIENT: Send: ESP32 TCP Client Data
I (2324) TCP-CLIENT: Send: ESP32 TCP Client Data
I (2524) TCP-CLIENT: Send: ESP32 TCP Client Data
I (2724) TCP-CLIENT: Send: ESP32 TCP Client Data
I (2724) TCP-CLIENT: Recv: Hello TCP Client
I (2924) TCP-CLIENT: Send: ESP32 TCP Client Data
I (2924) TCP-CLIENT: Recv: Hello TCP Client

```

6.3 Normal Log — TCP Client

```

I (2856) TCP-CLIENT: Got IP: 192.168.11.100
I (2858) TCP-CLIENT: WiFi connected
I (2860) TCP-CLIENT: TCP socket created
I (2862) TCP-CLIENT: TCP server connected
I (2864) TCP-CLIENT: Send: ESP32 TCP Client Data
I (2866) TCP-CLIENT: Recv: Hello from TCP Server!

```

```

I (1704) TCP-CLIENT: Got IP: 192.168.11.181
I (1704) TCP-CLIENT: WiFi connected
I (1704) TCP-CLIENT: TCP socket created
I (1714) main_task: Returned from app_main()
I (1724) wifi:<ba-add>idx:1 (ifx:0, 80:8f:1d:20:9d:f7), tid:0, ssn:
0, winSize:64
I (1724) TCP-CLIENT: TCP server connected
I (1724) TCP-CLIENT: Send: ESP32 TCP Client Data
I (1924) TCP-CLIENT: Send: ESP32 TCP Client Data
I (2124) TCP-CLIENT: Send: ESP32 TCP Client Data
I (2324) TCP-CLIENT: Send: ESP32 TCP Client Data
I (2524) TCP-CLIENT: Send: ESP32 TCP Client Data
I (2724) TCP-CLIENT: Send: ESP32 TCP Client Data
I (2724) TCP-CLIENT: Recv: Hello TCP Client
I (2924) TCP-CLIENT: Send: ESP32 TCP Client Data
I (2924) TCP-CLIENT: Recv: Hello TCP Client

```

7. Common Problems

Symptom	Cause	Solution
<code>connect()</code> returns -1, connection failed	Server not started or wrong IP	Confirm PC TCP Server is listening, IP and port are correct
Server <code>accept()</code> blocks and doesn't return	No client connected	Normal behavior, it's a blocking call

Symptom	Cause	Solution
Client <code>recv()</code> returns 0	Server actively closed connection	Normal disconnect, <code>break</code> then <code>close()</code>
Consecutive <code>send()</code> failures	Peer closed / network interrupted	TCP connection broken, need to <code>connect()</code> again
<code>bind()</code> failed	Port in use	Use a different port / wait 2 minutes (TIME_WAIT timeout)
<code>recv()</code> data "sticks" together	TCP is byte stream, no message boundaries	Define frame header+length or delimiter protocol at application layer
Server can only connect one client	Single-threaded sequential processing	Change to <code>xTaskCreate</code> for each <code>accept()</code> for independent tasks
Meaning of <code>listen()</code> second parameter <code>5</code>	Not max connections, it's the pending connection queue length	Already-accepted connections don't count